

Project 2: WannaCry
Christopher Protheroe &
Anthony Costa
12/12/24

Task 1: File Analysis

Use any tool of your choice to answer the following questions:

1. What is the program file size in bytes?

The size of the program in bytes is:

```
(kali㉿kali)-[~/Downloads]  
$ wc -c < wannacry  
3723264
```

2. What is the program's MD5, SHA1, and SHA256 hash?

The MD5 hash of the file is:

```
(kali㉿kali)-[~/Downloads]  
$ md5sum wannacry  
db349b97c37d22f5ea1d1841e3c89eb4  wannacry
```

The SHA hash values are:

```
(kali㉿kali)-[~/Downloads]  
$ shasum wannacry  
e889544aff85ffaf8b0d0da705105dee7c97fe26  wannacry  
  
(kali㉿kali)-[~/Downloads]  
$ sha256sum wannacry  
24d004a104d4d54034dbcffc2a4b19a11f39008a575aa614ea04703480b1022c  wannacry  
  
(kali㉿kali)-[~/Downloads]  
$ sha512sum wannacry  
d6c60b8f22f89cbd1262c0aa7ae240577a82002fb149e9127d4edf775a25abcda4e585b6113e7  
9ab4a24bb65f4280532529c2f06f7ffe4d5db45c0caf74fea38  wannacry
```

3. What does VirusTotal report when searching by its SHA3256 hash?

Project 2: WannaCry
Christopher Protheroe &
Anthony Costa
12/12/24

Virus total reports that 70/72 security vendors will flag the file as malicious.

70 / 72 security vendors flagged this file as malicious

Community Score: 1763

24d004a104d4d54034dbcf2a4b19a11f9908a575aa614ea04703480b1022c
hldrui.exe

Size: 3.55 MB | Last Analysis Date: 10 hours ago

peexe detect-debug-environment exploit direct-cpu-clock-access macro-create-cde checks-network-adapters long-sleeps malware cve-2017-0147 checks-user-input runtime-modules cve-2017-0144

DETECTION DETAILS RELATIONS ASSOCIATIONS BEHAVIOR COMMUNITY 30+

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Popular threat label: trojan.wannacry/wanna Threat categories: trojan ransomware worm Family labels: wannacry wanna wannacryptor

Security vendors' analysis

Vendor	Analysis
Acronis (Static ML)	Suspicious
Alibaba	Ransom.Win32/WannaCry_398
ALYac	Trojan.Ransom.WannaCrytor
Arcabit	Trojan.Ransom.WannaCrytor.H
AVG	SF-WNCryLdr-A [Trj]
Baidu	Win32.Worm.RBot.a
AhnLab-V3	Trojan.Win32.WannaCrytor.R200572
AliCloud	Exp-Win/CVE.2017.0147
Antiy-AVL	Trojan(Ransom)/Win32.Wanna
Avast	SF-WNCryLdr-A [Trj]
Avira (no cloud)	TR/Ransom.IZ
BitDefender	Trojan.Ransom.WannaCrytor.H

Task 2: Static Analysis

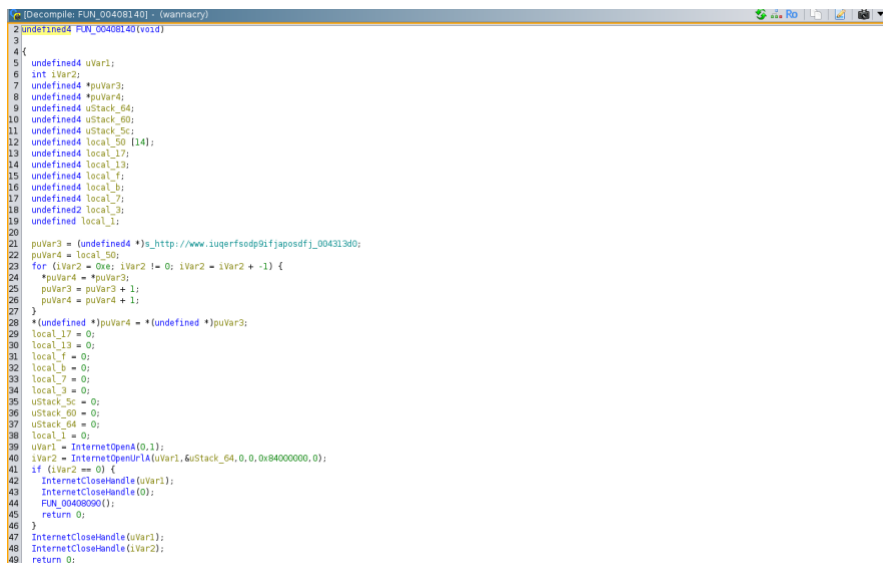
Answer the following questions using Ghidra:

1. Research and document (using screenshots and description) how WannaCry decides whether to infect your computer with ransomware.

Wannacry exploited a vulnerability in SMB protocol in older versions of Microsoft windows to infect systems on a network. Wannacry would scan the networks to see if there were exposed SMB ports on computers in the network (port 445). Microsoft released a patch for this exploit, MS17-010, however, this patch was not implemented on all machines. A machine downloaded this vulnerability through phishing or forced file uploading. Once on the machine the file would automatically execute itself. In order to continue the spreading of the ransomware a vulnerable system would mass ARP span the network and find all in use IP addresses, once it knows that information it would attempt SMB requests with that machine to spread its ransomware through the network. When Wannacry is first on a system it is a single .exe file labeled wannacry. As soon as execution begins there is an unpacking of resources inside this file (mainly resource 1831 and resource 2058), causing more files to be released on the system and more code to be executed. Before any malicious execution begins, Wannacry checks to see if its “kill-switch” domain has been

Project 2: WannaCry
Christopher Protheroe &
Anthony Costa
12/12/24

registered or not. If it has been, then Wannacry will abort execution and not infect the system, however if the domain is not registered then infection and encryption will begin. Or if the network is offline and is unable to generate a proper DNS response from the domain then it will begin infection. Wannacry binds itself to a system process on startup. This system process is typically labeled “Microsoft Security Center (2.0) Service”. Once it is bound to startup, the code execution will then edit register keys, both for the local user, and the system as a whole. The execution will then grant access to “All permissions” for all users, effectively sealing the ransomware's effectiveness. And this is the beginning because the encryption process has not yet started. There are many layers to the encryption process that will begin in parallel with other functional executions. For example, the choosing of a bitcoin address (1 of 3), is done by a random number generator based on the seeding of the random() function with the computers name as the seed for generation. The code also contains multiple .onion server addresses. These servers will be pinged and send information about the infected system.



```
[Decompile: FUN_00408140] - (wannacry)
2 undefined4 FUN_00408140(void)
3
4 {
5     undefined4 uVar1;
6     int iVar2;
7     undefined4 *puVar3;
8     undefined4 *puVar4;
9     undefined4 uStack_64;
10    undefined4 uStack_60;
11    undefined4 uStack_5c;
12    undefined4 local_50 [14];
13    undefined4 local_17;
14    undefined4 local_13;
15    undefined4 local_f;
16    undefined4 local_b;
17    undefined4 local_7;
18    undefined2 local_3;
19    undefined local_1;
20
21    puVar3 = (undefined4 *)http://www.iuqerfsodp9ifjaposdfjj_00431340;
22    puVar4 = local_50;
23    for (iVar2 = 0; iVar2 != 0; iVar2 = iVar2 + -1) {
24        *puVar4 = *puVar3;
25        puVar3 = puVar3 + 1;
26        puVar4 = puVar4 + 1;
27    }
28    *(undefined *)puVar4 = *(undefined *)puVar3;
29    local_17 = 0;
30    local_13 = 0;
31    local_f = 0;
32    local_b = 0;
33    local_7 = 0;
34    local_3 = 0;
35    uStack_5c = 0;
36    uStack_60 = 0;
37    uStack_64 = 0;
38    local_1 = 0;
39    uVar1 = InternetOpenA(0,1);
40    iVar2 = InternetOpenUrlA(uVar1,6uStack_64,0,0,0,0x84000000,0);
41    if (iVar2 == 0) {
42        InternetCloseHandle(uVar1);
43        InternetCloseHandle(0);
44        FUN_00408090();
45        return 0;
46    }
47    InternetCloseHandle(uVar1);
48    InternetCloseHandle(iVar2);
49    return 0;
50 }
```

Here in the WinMain for the WannaCry.exe in Ghidra you can see the kill switch URL, that a connection attempt is made to see if the domain is registered and therefore would terminate the execution of the rest of the ransomware.

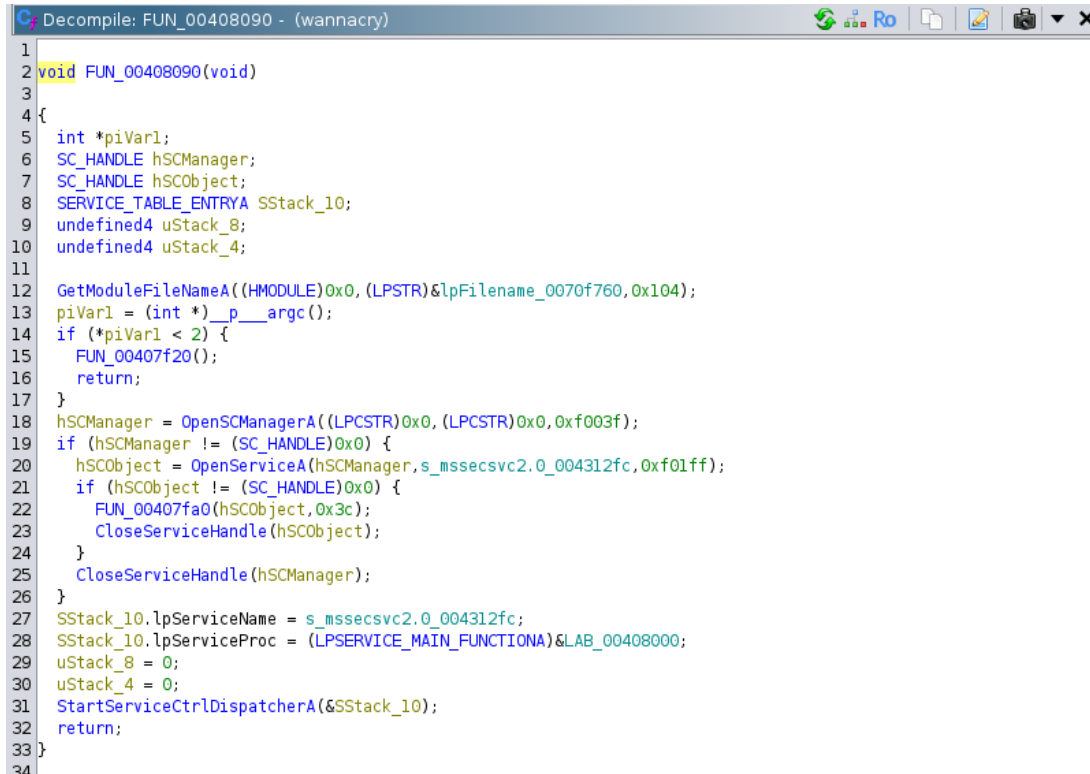
2. What is the entry point (address in program) of where the malicious code begins?

Specifically, which function/address does it call when it decides to infect your

Project 2: WannaCry
Christopher Protheroe &
Anthony Costa
12/12/24

computer?

Fun_00408090(); memory address: 00408090 is considered the true entry, or start of the ransomware execution. This Main function is not the start because if the kill-switch domain is active then the program will abort execution. If the domain is not registered, then the ransomware will begin to infect the computer/device/network.

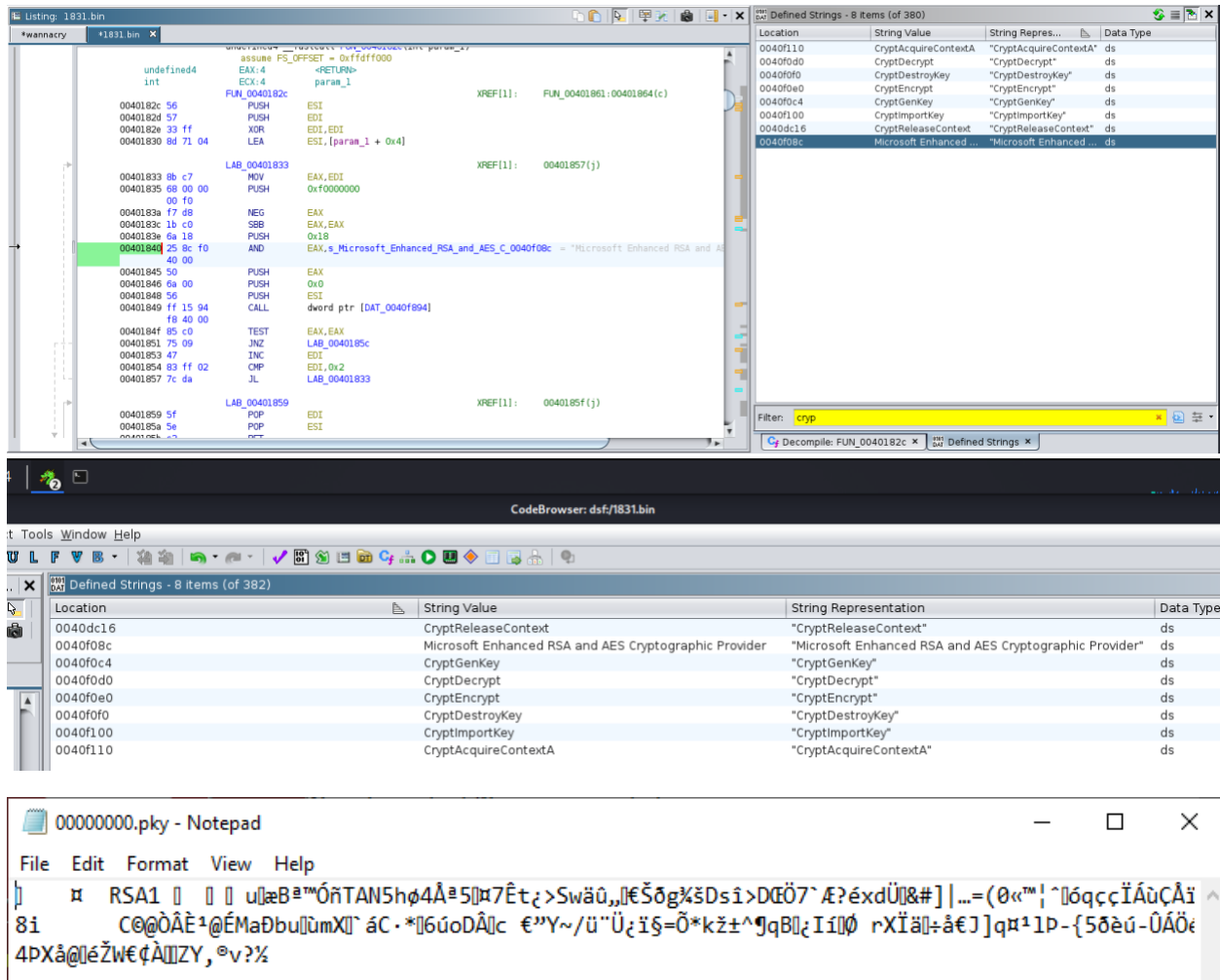


```
Decompile: FUN_00408090 - (wannacry)
1
2 void FUN_00408090(void)
3
4 {
5     int *piVar1;
6     SC_HANDLE hSCManager;
7     SC_HANDLE hSCObject;
8     SERVICE_TABLE_ENTRYA SStack_10;
9     undefined4 uStack_8;
10    undefined4 uStack_4;
11
12    GetModuleFileNameA((HMODULE)0x0, (LPSTR)&lpFilename_0070f760, 0x104);
13    piVar1 = (int *)__p__argc();
14    if (*piVar1 < 2) {
15        FUN_00407f20();
16        return;
17    }
18    hSCManager = OpenSCManagerA((LPCSTR)0x0, (LPCSTR)0x0, 0xf003f);
19    if (hSCManager != (SC_HANDLE)0x0) {
20        hSCObject = OpenServiceA(hSCManager, s_mssecsvc2.0_004312fc, 0xf01ff);
21        if (hSCObject != (SC_HANDLE)0x0) {
22            FUN_00407fa0(hSCObject, 0x3c);
23            CloseServiceHandle(hSCObject);
24        }
25        CloseServiceHandle(hSCManager);
26    }
27    SStack_10.lpServiceName = s_mssecsvc2.0_004312fc;
28    SStack_10.lpServiceProc = (LPSERVICE_MAIN_FUNCTIONA)&LAB_00408000;
29    uStack_8 = 0;
30    uStack_4 = 0;
31    StartServiceCtrlDispatcherA(&SStack_10);
32    return;
33 }
34
```

3. What encryption algorithm does it use to encrypt? Symmetric or Asymmetric? What specific mode? ECB, CBC, CTR, GCM, etc.

a. Document using screenshots of where you found this answer.

Project 2: WannaCry
Christopher Protheroe &
Anthony Costa
12/12/24



In the decompiled code there are numerous cryptographic windows API calls. As seen above there is a call to the Microsoft Enhanced RSA and AES Cryptographic Provider.

CryptGenKey is used to generate an RSA key pair.

CryptEncrypt is used for encrypting the files using the symmetric AES key

CryptImportKey is used to import the attackers public key

CryptExportKey is used for exporting the now encrypted AES keys using the

CryptDestroyKey is used to destroy the memory area where the key was stored.

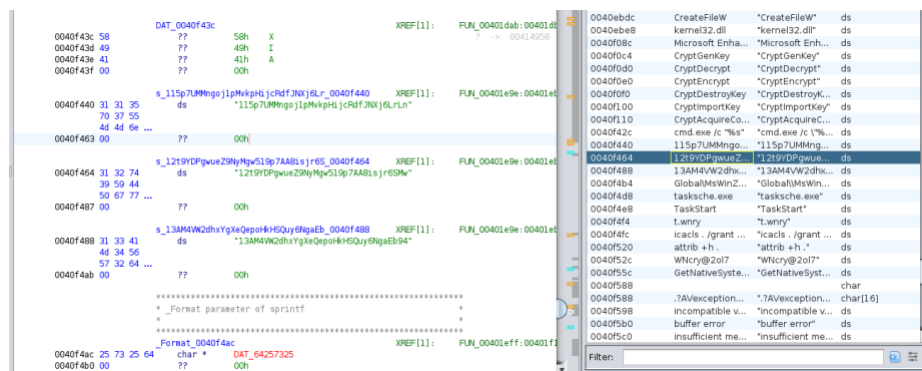
There are 3 explicit keys present in the ransomware (not counting the ones created for each file). Victim's private and public key which are 2048-bit RSA key pairs. And 2 Wannacry ransomware AES 128 keys, only one key is shared with the victim, the Wannacry

Project 2: WannaCry
Christopher Protheroe &
Anthony Costa
12/12/24

AES public key. The victim's private key will be encrypted using the attacking public AES key and will be stored in the file above. After this the original victim's private key will get destroyed. Wannacry will then generate unique AES 128 keys for each file, using a function that generates AES 128 keys. The ransomware will then encrypt all AES keys using the victim's public key. The attacking private key is never transmitted in these files, so it is seemingly impossible to decrypt. The use of creating a new key for every file plays a vital role in malware, due to the nature that each file has its own key. In case there was memory slippage or leaking the leaked key would now only decrypt one file not all of them.

4. Where can you find the bitcoin address to pay?

a. Find the bitcoin address string in the program.



```
1 void randomly_select_bitcoin_addr(void)
2
3 {
4     bool bVar1;
5     undefined3 extraout_var;
6     int randomnumber;
7     char unknownbitarray [780];
8     char *bitcoinaddresses [3];
9
10    bitcoinaddresses[0] = s_13AM4VW2dhxYgXeQepoHkHSQuy6NgaEb_0040f488;
11    bitcoinaddresses[1] = s_12t9YDPgwueZ9NyMgw519p7AA8isjr6S_0040f464;
12    bitcoinaddresses[2] = s_115p7UMMngoJlpMvkpHiJcRdfJNXj6Lr_0040f440;
13    bVar1 = read_or_write_to_c.wnry(unknownbitarray,1);
14    if (CONCAT31(extraout_var,bVar1) != 0) {
15        randomnumber = rand();
16        strcpy(unknownbitarray + 0xb2,bitcoinaddresses[randomnumber % 3]);
17        read_or_write_to_c.wnry(unknownbitarray,0);
18    }
19    return;
20 }
```

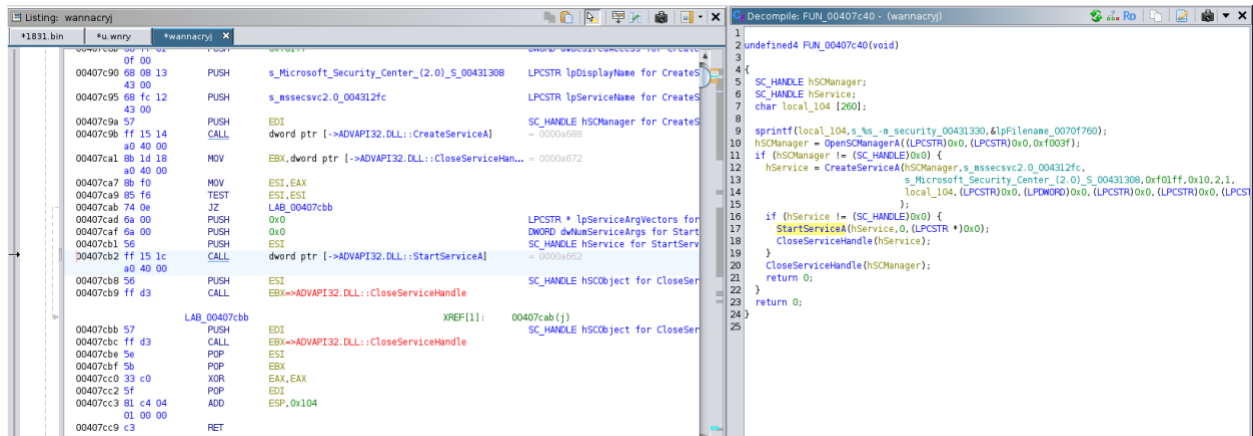
There are 3 bitcoin addresses inside the program and function that chooses which one that the infected system will have to send money to. The choice is from a random number generated with mod 3, to get options 0,1,2.

Project 2: WannaCry
Christopher Protheroe &
Anthony Costa
12/12/24

They are found in the exported file from Ghidra that is password protected when decrypted, at these memory addresses:

1. 0040f43f s_115p7UMMngoj1pMvvpHijcRdfJNXj6Lr_0040f440
 - a. 0040f440 "115p7UMMngoj1pMvvpHijcRdfJNXj6LrLn"
2. 0040f463 s_12t9YDPgwueZ9NyMgw519p7AA8isjr6S_0040f464
 - a. 0040f464 "12t9YDPgwueZ9NyMgw519p7AA8isjr6SMw"
3. 0040f487 s_13AM4VW2dhxYgXeQepoHkHSQuy6NgaEb_0040f488
 - a. 0040f488 13AM4VW2dhxYgXeQepoHkHSQuy6NgaEb94"

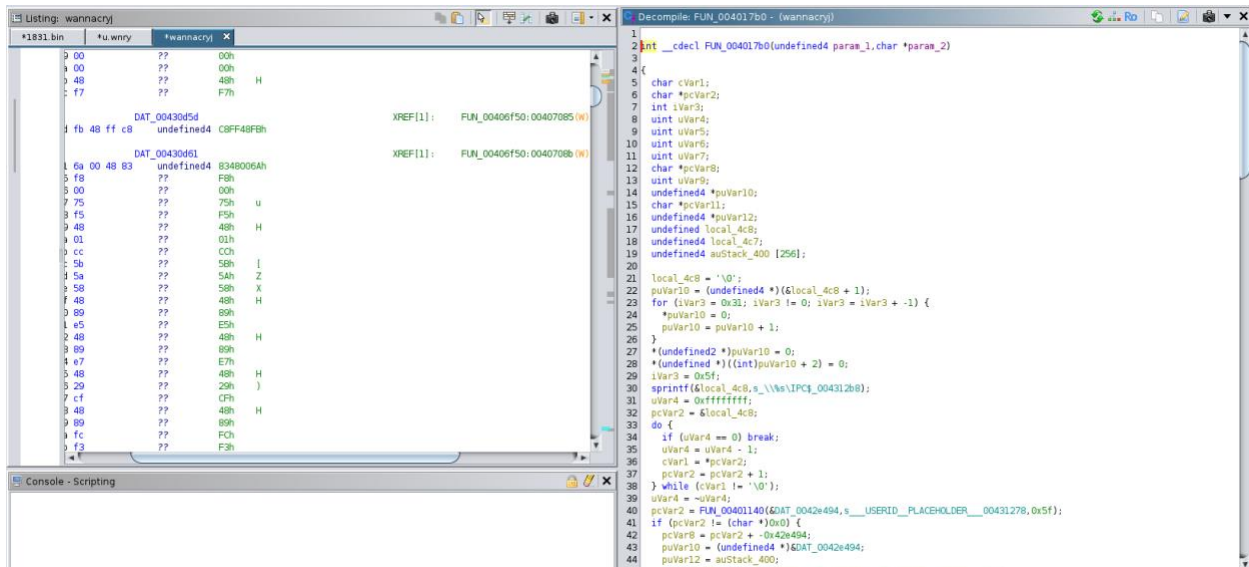
5. WannaCry leverages the Eternal Blue exploit of the SMB protocol to worm itself to other computers in a network. Find and document where this code exists inside the program.



```
1  undefined4 FUN_00407c40(void)
2
3
4  {
5      SC_HANDLE hServiceManager;
6      SC_HANDLE hService;
7      char local_104 [200];
8
9      sprintf(local_104,s_115p7UMMngoj1pMvvpHijcRdfJNXj6LrLn);
10     hServiceManager = OpenSCManagerA(LPCSTR)0x0,(LPCSTR)0x0,0xf003f);
11     if (hServiceManager != (SC_HANDLE)0x0) {
12         hService = CreateServiceA(hServiceManager,s_12t9YDPgwueZ9NyMgw519p7AA8isjr6SMw,
13                                 s_Microsoft_Security_Center_(2.0)_S_00431308,0xf01ff,0x10,2,1,
14                                 local_104,(LPCSTR)0x0,(LPCSTR)0x0,(LPCSTR)0x0,(LPCSTR)0x0,(LPCSTR)0x0);
15     }
16     if (hService != (SC_HANDLE)0x0) {
17         StartServiceA(hService,0,(LPCSTR *)0x0);
18         CloseServiceHandle(hService);
19     }
20     CloseServiceHandle(hServiceManager);
21     return 0;
22 }
23
24
25
```

This is where the process is bound on a computer under process tasksche.exe namely "mssecvsvc2.0", namely "Microsoft Security Center 2.0".

Project 2: WannaCry
Christopher Protheroe &
Anthony Costa
12/12/24



Using the path `\\%s\\IPC$` allows you to establish IPC connections over SMB, enabling processes on different machines to communicate with each other. This is done through named pipes, which provide a reliable way to send and receive data over TCP port 445 (SMB).

Task 3: Dynamic Analysis

Answer the following questions using x64dbg and your answers from Task 2:

2. Open the WannaCry program in x64dbg and confirm using dynamic analysis:

- a. Set a breakpoint on the function responsible for reaching out to the “interesting looking” URL. Does this URL in the debugger match what you found from static analysis? (screenshot and discuss)

Yes, it matches, ghidra’s decompiled display of the URL did not contain enough characters but if you look at the assembly in ghidra then the links are the same.

Project 2: WannaCry
Christopher Protheroe &
Anthony Costa
12/12/24

```
00408140 83EC 50      sub esp,50
00408143 56          push esi
00408144 57          push edi
00408145 B9 0E000000 mov ecx,E
0040814A BE D0134300 mov esi,wannacry.4313D0
0040814F 8D7C24 08   lea edi,dword ptr ss:[esp+8]
00408153 33C0        xor eax,eax
00408155 F3:A5       rep movsd
00408157 A4          movsb
00408158 894424 41   mov dword ptr ss:[esp+41],eax
0040815C 894424 45   mov dword ptr ss:[esp+45],eax
00408160 894424 49   mov dword ptr ss:[esp+49],eax
00408164 894424 4D   mov dword ptr ss:[esp+4D],eax
00408168 894424 51   mov dword ptr ss:[esp+51],eax
0040816C 66:894424 55 mov word ptr ss:[esp+55],ax
00408171 50          push eax
00408172 50          push eax
00408173 50          push eax
```

esi=77A26A68 "LdrpInitializeProcess"
004313D0 "http://www.iuqerfsodp9ifjaposdfjhgosurijfaewrgwea.com"
.text:0040814A wannacry:\$814A #814A

b. Set a breakpoint on the function(s) responsible for setting the encryption key to encrypt files. Confirm using the debugger and Windows Crypto API documentation how it encrypts files. (Algorithm, mode, asymmetric/symmetric)

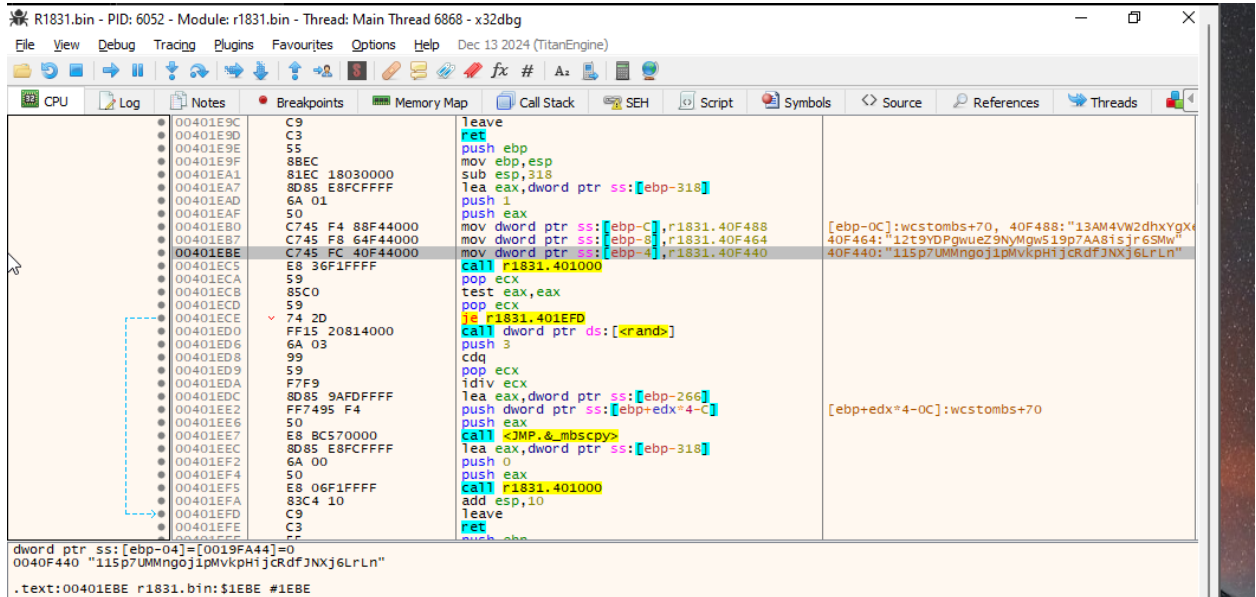
```
004017FF 56          push esi
00401800 8BF1        mov esi,ecx
00401802 E8 14000000 test byte ptr ss:[esp+8],1
00401807 F64424 08 01 je r1831.401815
0040180C 74 07       push esi
0040180E 56          call <JMP.&void __cdecl operator delete(void*)>
0040180F E8 D45E0000 pop ecx
00401814 59          mov eax,esi
00401815 8BC6        pop esi
00401817 5E          ret 4
00401818 C701 EC814000 mov dword ptr ds:[ecx],r1831.4081EC
00401821 83C1 10     add ecx,10
00401824 51          push ecx
00401825 FF15 3C804000 call dword ptr ds:[<DeleteCriticalSection>]
00401828 C3          ret
0040182C 56          push esi
0040182D 57          push edi
0040182E 33FF        xor edi,edi
00401830 8D71 04     lea esi,dword ptr ds:[ecx+4]
00401833 8BC7        mov eax,edi
00401835 68 000000F0 push F0000000
0040183A F7D8        neg eax
0040183C 1BC0        sbb eax,eax
0040183E 6A 18       push 18
00401840 25 8CF04000 and eax,r1831.40F08C
00401845 50          push eax
00401846 6A 00       push 0
00401848 56          push esi
00401849 FF15 94F84000 call dword ptr ds:[40F894]
0040184F 85C0        test eax,eax
00401851 75 09       jne r1831.40185C
```

esi:"LdrpInitializeProcess"
esi:"LdrpInitializeProcess"
esi:"LdrpInitializeProcess"
esi:"LdrpInitializeProcess"
esi:"LdrpInitializeProcess"
esi:"LdrpInitializeProcess"
edi:"minkernel\\ntdll\\ldrinit.c"
edi:"minkernel\\ntdll\\ldrinit.c"
esi:"LdrpInitializeProcess"
edi:"minkernel\\ntdll\\ldrinit.c"
esi:"LdrpInitializeProcess"
40F08C:"Microsoft Enhanced RSA and AES Cryptographic Provider"
esi:"LdrpInitializeProcess"
edi:"minkernel\\ntdll\\ldrinit.c"

eax=0
0040F08C "Microsoft Enhanced RSA and AES Cryptographic Provider"
.text:00401840 r1831.bin:1840 #1840

c. Set a breakpoint on the function which displays the bitcoin address and confirm it matches what you expected from static analysis.

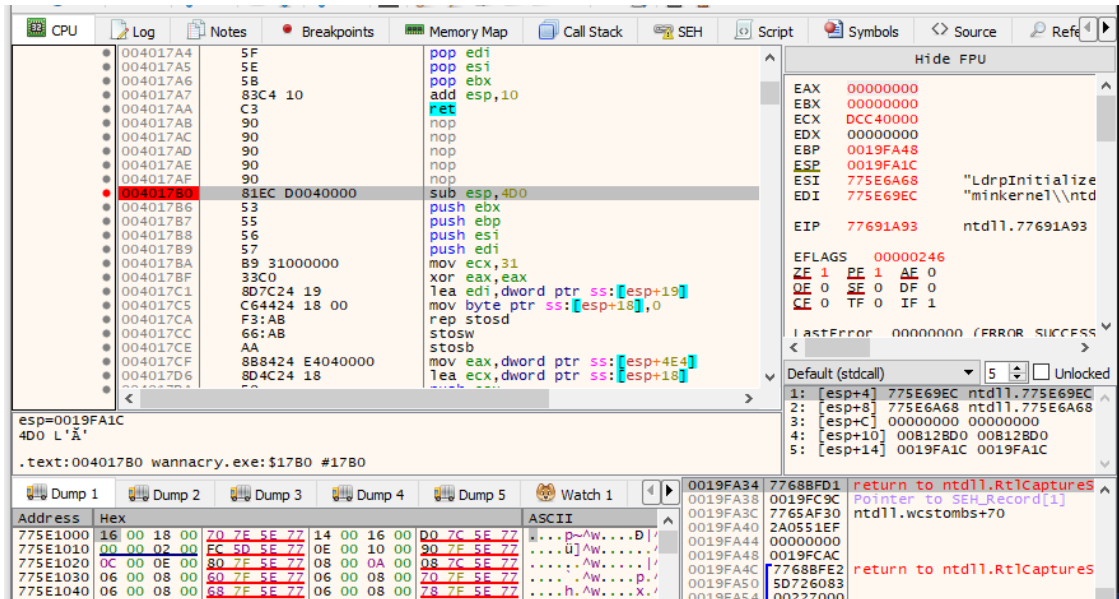
Project 2: WannaCry
Christopher Protheroe &
Anthony Costa
12/12/24



Yes it matches. Like in the static analysis there are 3 bitcoin addresses, as the program progresses, it will randomly generate a number mod it by 3 and then choose 1 of the 3 wallet addresses to display on the ransomware note.

- d. Set a breakpoint on the function responsible for checking whether a remote computer is vulnerable to Eternal Blue
 - i. Specifically, the function which initiates an SMB v1 connection to a remote computer.

Project 2: WannaCry
Christopher Protheroe &
Anthony Costa
12/12/24



These function and the nested dll function calls within.

Task 4: Network Analysis

7. Answer the following questions using Wireshark:

a. Show the DNS lookup request to the strange looking URL.

No.	Time	Source	Destination	Protocol	Length	Info
34	23.018047777	192.168.240.128	192.168.240.128	ICMP	100	Destination unreachable (Port unreachable)
35	25.017877384	192.168.240.129	192.168.240.128	DNS	78	Standard query 0x8e76 A wpad.localdomain
36	25.017906525	192.168.240.128	192.168.240.129	ICMP	100	Destination unreachable (Port unreachable)
37	55.301700880	192.168.240.129	192.168.240.128	DNS	111	Standard query 0xd119 A www.iuqerfsodp9ifjaposdfjhgosurijfaewrgwea.com
38	55.301736080	192.168.240.128	192.168.240.129	ICMP	100	Destination unreachable (Port unreachable)
39	56.299760017	192.168.240.129	192.168.240.128	DNS	111	Standard query 0xd119 A www.iuqerfsodp9ifjaposdfjhgosurijfaewrgwea.com
40	56.299760017	192.168.240.128	192.168.240.129	ICMP	100	Destination unreachable (Port unreachable)
41	57.299410709	192.168.240.129	192.168.240.128	DNS	111	Standard query 0xd119 A www.iuqerfsodp9ifjaposdfjhgosurijfaewrgwea.com
42	57.299434220	192.168.240.128	192.168.240.129	ICMP	100	Destination unreachable (Port unreachable)
43	59.327647079	192.168.240.129	192.168.240.128	DNS	111	Standard query 0xd119 A www.iuqerfsodp9ifjaposdfjhgosurijfaewrgwea.com
44	59.327676879	192.168.240.128	192.168.240.129	ICMP	100	Destination unreachable (Port unreachable)

DNS lookup request to the URL (killswitch). Fails because there is no network connection on either machine to actually receive a message back from the domain that it is active.

b. How does the victim windows VM scan the network for other computers to Infect?

Project 2: WannaCry
Christopher Protheroe &
Anthony Costa
12/12/24

167	144.426507309	VMware_0e:01:41	ARP	62 Who has 192.168.240.57 Tell 192.168.240.129
168	144.426507919	VMware_0e:01:41	ARP	62 Who has 192.168.240.67 Tell 192.168.240.129
169	144.426507979	VMware_0e:01:41	ARP	62 Who has 192.168.240.77 Tell 192.168.240.129
170	144.426508019	VMware_0e:01:41	ARP	62 Who has 192.168.240.87 Tell 192.168.240.129
171	144.426508059	VMware_0e:01:41	ARP	62 Who has 192.168.240.97 Tell 192.168.240.129
172	144.426508089	VMware_0e:01:41	ARP	62 Who has 192.168.240.107 Tell 192.168.240.129
173	144.4265075521	VMware_0e:01:41	ARP	62 Who has 192.168.240.117 Tell 192.168.240.129
174	144.4265075591	VMware_0e:01:41	ARP	62 Who has 192.168.240.157 Tell 192.168.240.129
175	144.4265075631	VMware_0e:01:41	ARP	62 Who has 192.168.240.167 Tell 192.168.240.129
176	144.4265075671	VMware_0e:01:41	ARP	62 Who has 192.168.240.177 Tell 192.168.240.129
177	144.426612011	VMware_0e:01:41	ARP	62 Who has 192.168.240.187 Tell 192.168.240.129
178	144.426612071	VMware_0e:01:41	ARP	62 Who has 192.168.240.197 Tell 192.168.240.129
179	144.426647832	VMware_0e:01:41	ARP	62 Who has 192.168.240.207 Tell 192.168.240.129
180	144.426647892	VMware_0e:01:41	ARP	62 Who has 192.168.240.217 Tell 192.168.240.129
181	144.426680072	VMware_0e:01:41	ARP	62 Who has 192.168.240.227 Tell 192.168.240.129
182	144.437384848	VMware_17:06:6a	ARP	44 Who has 192.168.240.17 Tell 192.168.240.128
183	144.437379339	VMware_c0:00:02	ARP	62 192.168.240.1 is at 00:50:56:c0:00:02
184	144.926272230	VMware_0e:01:41	ARP	62 Who has 192.168.240.127 Tell 192.168.240.129
185	144.926272620	VMware_0e:01:41	ARP	62 Who has 192.168.240.137 Tell 192.168.240.129

Mass ARP requests to see what responds to the infected machine to then see who to try and initiate and SMB request with.

c. Show the infected Windows VM attempt an SMB connection to the Linux VM.

No.	Time	Source	Destination	Protocol	Length	Info
418	84.114244433	192.168.240.129	192.168.240.128	TCP	56	445 -> 49851 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
419	84.114260393	192.168.240.128	192.168.240.129	TCP	56	445 -> 49851 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
421	84.029060184	192.168.240.129	192.168.240.128	TCP	68	[TCP Port numbers reused] 49851 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
422	84.029061405	192.168.240.128	192.168.240.129	TCP	56	445 -> 49851 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
447	85.129257785	192.168.240.129	192.168.240.128	TCP	68	[TCP Port numbers reused] 49851 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
448	85.129278376	192.168.240.128	192.168.240.129	TCP	56	445 -> 49851 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1610	1197.8991404	192.168.240.128	192.168.240.129	TCP	56	445 -> 49848 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1635	1198.40956473	192.168.240.129	192.168.240.128	TCP	68	[TCP Port numbers reused] 49848 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1636	1198.40956606	192.168.240.128	192.168.240.129	TCP	56	445 -> 49848 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1654	1198.90950440	192.168.240.129	192.168.240.128	TCP	68	[TCP Port numbers reused] 49848 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1655	1198.90950193	192.168.240.128	192.168.240.129	TCP	56	445 -> 49848 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
2057	1325.8799796	192.168.240.129	161.254.42.170	TCP	68	54376 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2058	1325.8801945	192.168.240.129	65.34.56.154	TCP	68	54383 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2059	1325.8802824	192.168.240.129	143.144.36.90	TCP	68	54384 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2060	1325.8802826	192.168.240.129	175.69.215.179	TCP	68	54385 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2061	1325.8803004	192.168.240.129	109.184.77.123	TCP	68	54386 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2062	1325.8804730	192.168.240.129	10.126.21.168	TCP	68	54387 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2063	1325.8805448	192.168.240.129	19.149.224.33	TCP	68	54388 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2064	1325.8806362	192.168.240.129	103.243.189.151	TCP	68	54391 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2065	1325.8804364	192.168.240.129	181.79.87.61	TCP	68	54392 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2066	1325.88051405	192.168.240.129	29.24.91.149	TCP	68	54395 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2067	1325.88051513	192.168.240.129	65.92.205.105	TCP	68	54396 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2068	1325.88053032	192.168.240.129	72.114.130.53	TCP	68	54397 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2069	1325.88054477	192.168.240.129	3.114.90.179	TCP	68	54399 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2070	1325.88074374	192.168.240.129	34.163.20.52	TCP	68	54404 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2071	1325.88075356	192.168.240.129	184.163.190.68	TCP	68	54405 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2072	1325.88076550	192.168.240.129	172.19.21.171	TCP	68	54406 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2073	1325.88077758	192.168.240.129	175.198.177.158	TCP	68	54407 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2074	1325.88079208	192.168.240.129	183.85.253.20	TCP	68	54410 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2075	1325.88080131	192.168.240.129	17.120.201.72	TCP	68	54411 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2076	1325.88079155	192.168.240.129	60.56.130.98	TCP	68	54417 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2077	1325.88079158	192.168.240.129	130.0.10.74	TCP	68	54410 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2078	1325.8808435	192.168.240.129	77.104.143.75	TCP	68	54410 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2079	1325.88084176	192.168.240.129	17.34.58.568	TCP	68	54410 -> 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM

Windows VM attempting an SMB request through TCP port 445 to multiple IP addresses, the first one being the Linux VM. Due to the fast-firing nature of these packets, my Wireshark picked up these attempted connections as TCP requests not SMB requests. They are still operating on TCP port 445, which is the correct SMB port to attack.

```
(kali㉿kali)-[~/Downloads]
$ strings wannacry | grep "IPC"
\\%s\IPC$
```